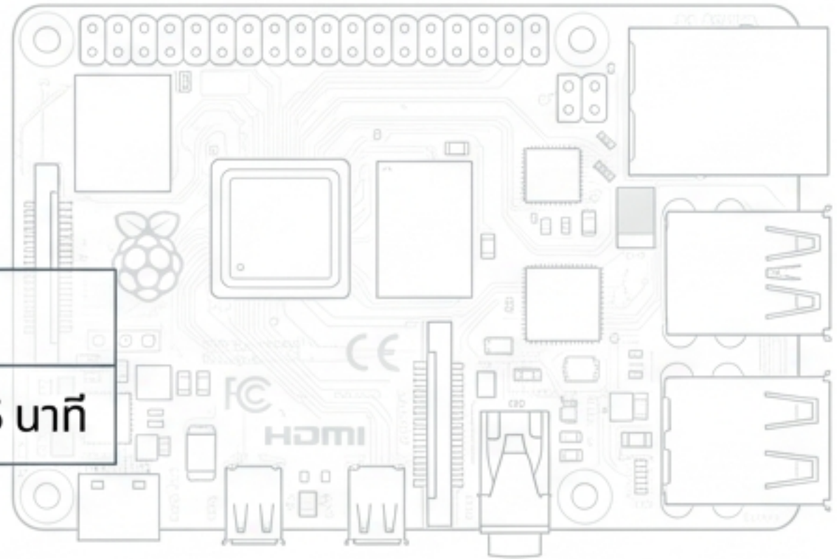
Web Dashboard & โครงงาน IoT (Weather Monitoring System)

การบูรณาการข้อมูลจากเซ็นเซอร์สู่หน้าเว็บแสดงผลแบบเรียลไทม์

บรรยายโดย ผศ. ดร. ชนันท์กรณ์ จันแดง

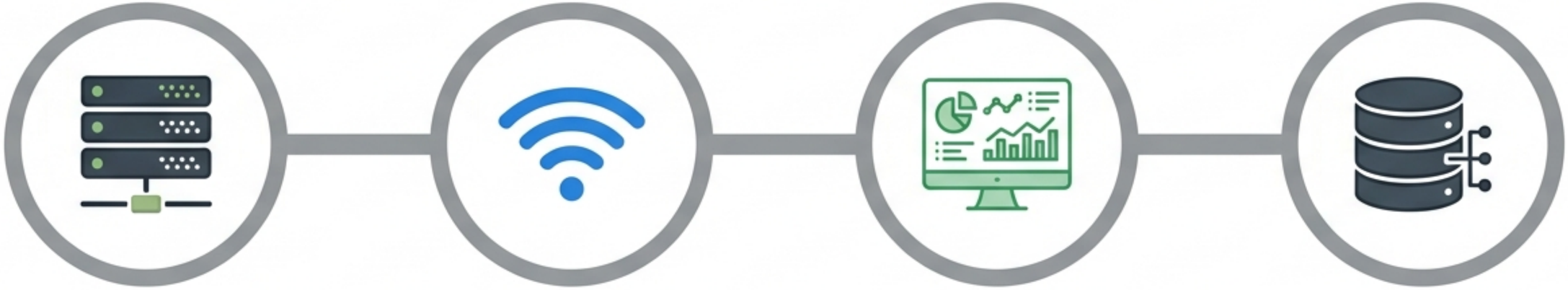


ส่วนที่ 4: การพัฒนา Web Dashboard และโครงงาน IoT (ระยะเวลา 3 ชั่วโมง)
รูปแบบการเรียน: บรรยาย 30 นาที + สาธิต 45 นาที + Workshop 90 นาที + นำเสนอ 15 นาที

NotebookLM

เป้าหมายของเวิร์กชอป (The End Goal)

Destination Map



ตั้งค่า Web Server บน Raspberry Pi ด้วย Flask

ดึงข้อมูลเซ็นเซอร์จาก ESP32 ผ่านเครือข่าย

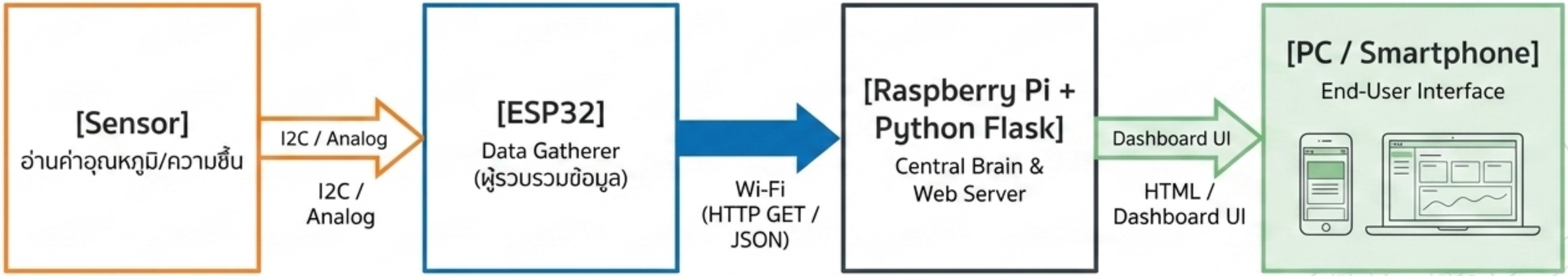
สร้าง Web Dashboard แสดงผลแบบเรียลไทม์

บันทึกและเรียกดูข้อมูลย้อนหลัง (CSV)

เมื่อจบส่วนนี้ คุณจะสามารถบูรณาการฮาร์ดแวร์และซอฟต์แวร์ทั้งหมดให้เป็นระบบ IoT ที่สมบูรณ์แบบ

NotebookLM

การเดินทางของข้อมูล (System Architecture)



NotebookLM

ขุมพลังเบื้องหลังหน้าเว็บ: Flask Framework

Python Logic

Web Browser

Lightweight
(เบาและทำงานเร็ว)

Python-Native
(เขียนด้วย Python ทั้งหมด)

IoT Ready
(เหมาะสำหรับรันบน Raspberry Pi)

```
$ pip3 install flask
```

NotebookLM

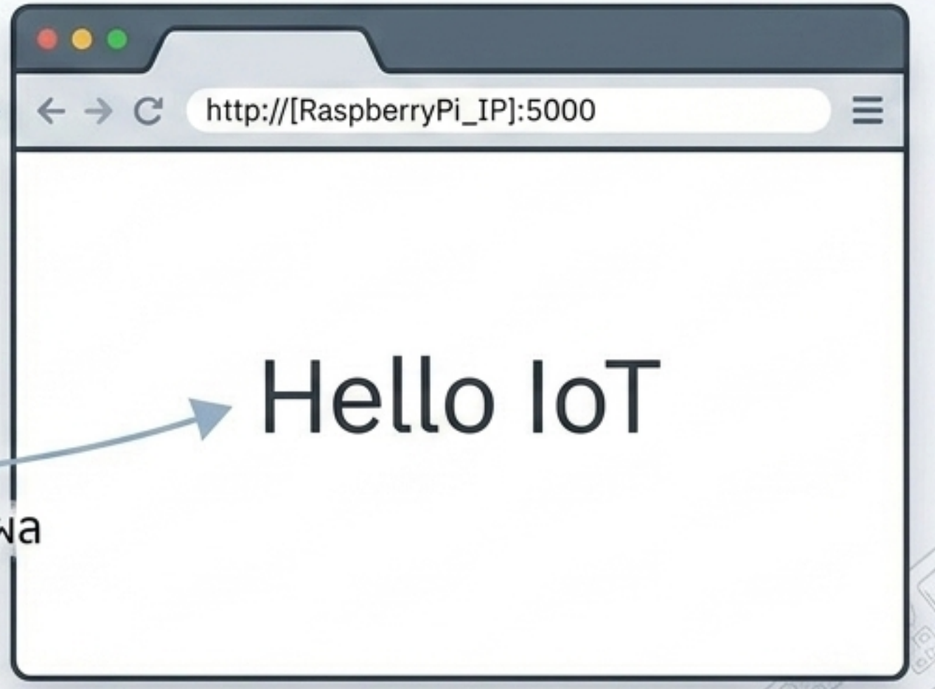
Workshop: Web Server แรกของเรา

```python
from flask import Flask
app = Flask(__name__)

@app.route("/")
def home():
    return "Hello IoT"

app.run(host="0.0.0.0")
```

1 กำหนด URL Path หรือ Routing

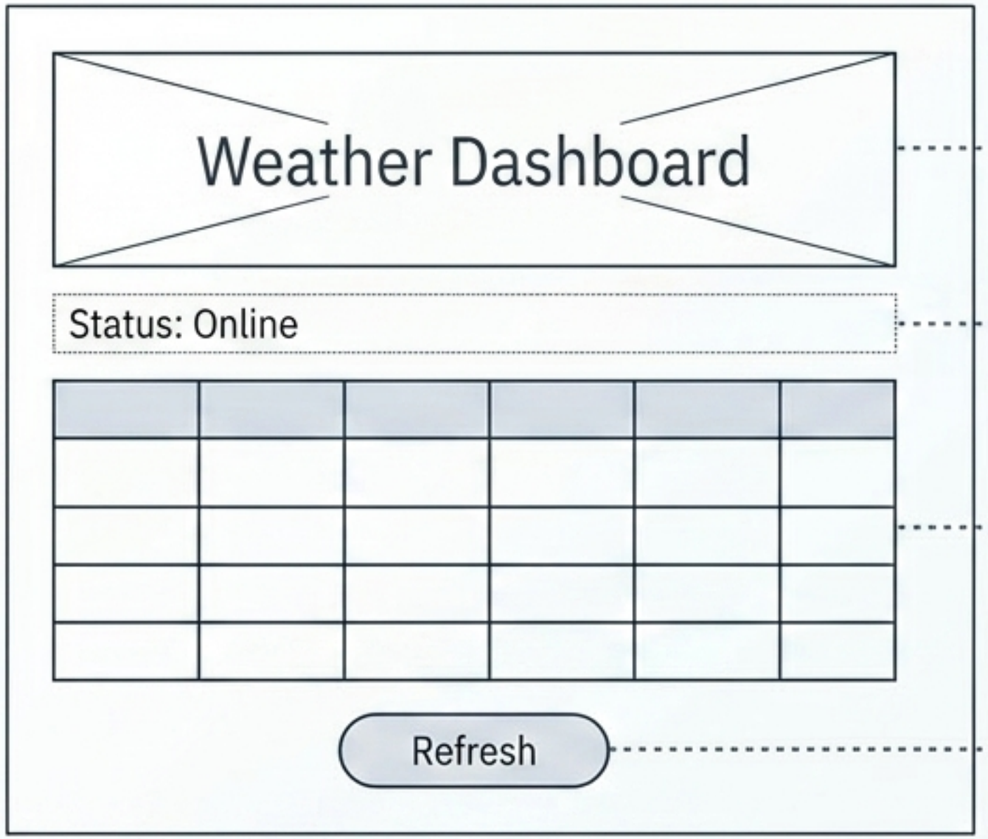


2 ข้อความที่จะส่งไปแสดงผล

3 เปิดรับการเชื่อมต่อจากทุกอุปกรณ์ในวง LAN

NotebookLM

การสร้างโครงร่างหน้าเว็บ (HTML Structure)



<H1> (Heading)

<P> (Paragraph)

<TABLE> (Table layout for data)

<BUTTON> (Interactive trigger)

Action Item / Workshop: สร้างไฟล์ HTML พื้นฐานสำหรับเตรียมรับข้อมูลอุณหภูมิและความชื้น

NotebookLM

เชื่อมโยงข้อมูล (Raspberry Pi ขอดูข้อมูลจาก ESP32)

Step 1 (Request): Pi ส่งคำสั่ง HTTP GET ผ่านไลบรารี requests

```
import requests
response = requests.get('http://[ESP32_IP]')
```

Step 2 (Response): ESP32 ตอบกลับมาเป็นข้อมูลรูปแบบ JSON

```
{"temperature": 30.5, "humidity": 72.0}
```

Step 3 (Extract): Pi แกะกล่องข้อมูล JSON เพื่อนำไปใช้ต่อ

```
data = response.json()
```

NotebookLM

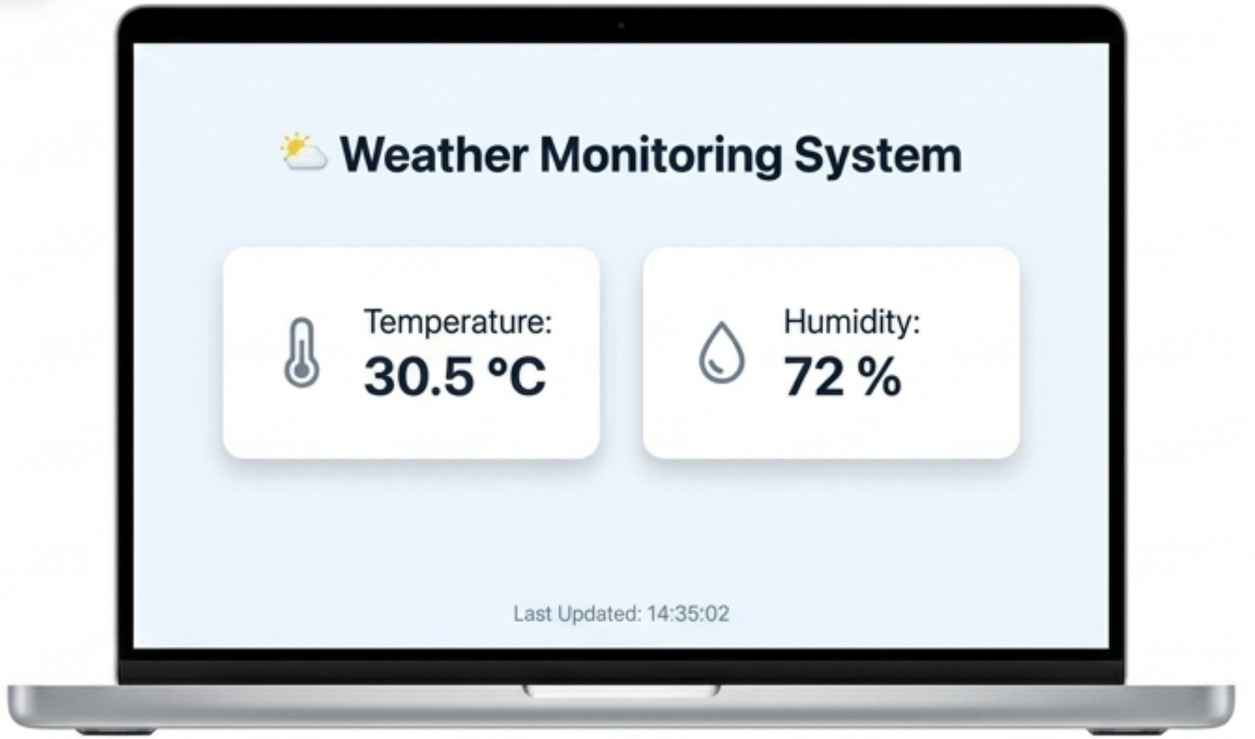
ประกอบร่าง Web Dashboard (Live Data Integration)

```
{
  {'temp':30.5}
}
```

+

```
<H1>
    Weather Dashboard
<P>
<DIV>
</DIV>
```

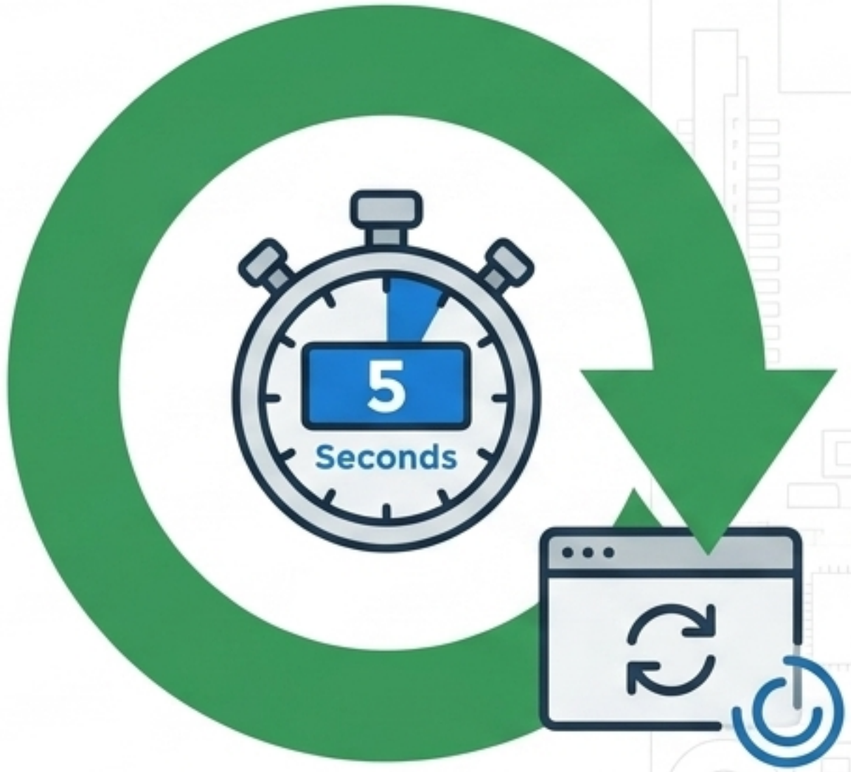
ข้อมูลดิบ (JSON) + โครงสร้างเว็บ (HTML)
= Dashboard ที่สมบูรณ์



NotebookLM

การอัปเดตแบบเรียลไทม์ (Auto Refresh)

หน้าเว็บที่อยู่นิ่งจะไม่เห็นการเปลี่ยนแปลง ต้องสั่งให้รีเฟรชตัวเอง



The Simple Method (HTML Meta)

```
<meta http-equiv="refresh"
content="5">
```

(ง่ายที่สุด รีเฟรชทั้งหน้าเว็บทุกๆ 5 วินาที)

The Advanced Method (JavaScript)

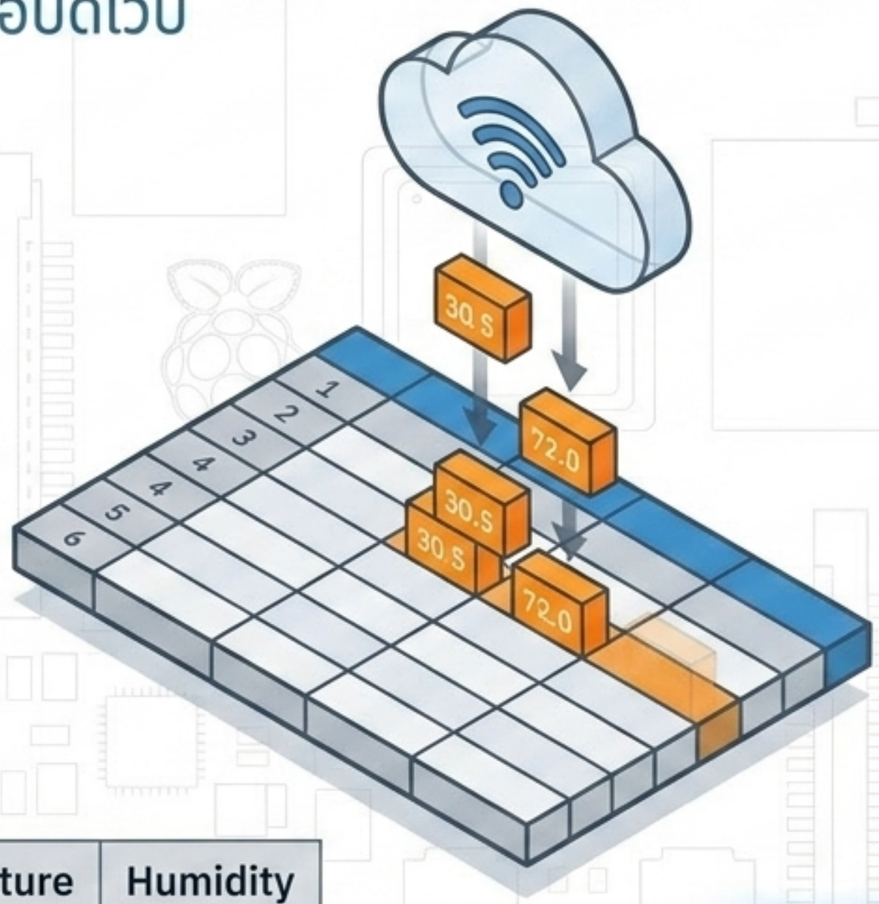
ดึงเฉพาะข้อมูลใหม่มาเปลี่ยนโดยไม่ต้องโหลดหน้าเว็บใหม่ทั้งหมด (AJAX/Fetch)

Workshop Test: ทริกเกอร์เซ็นเซอร์ (เป่าลม/เพิ่มความร้อน) และสังเกตการเปลี่ยนแปลงบนหน้าจออัตโนมัติ

NotebookLM

หน่วยความจำระยะยาว (บันทึกข้อมูลลง CSV)

ข้อมูลแบบเรียลไทม์จะหายไปเมื่อปิดเว็บ
เราต้องบันทึกเพื่อดูย้อนหลัง



File Structure (weather.csv)

	Date	Time	Temperature	Humidity
1	2023-10-25	10:00:00	30.5	72.0
2	2023-10-25	10:00:05	31.0	70.5

Code Metaphor: ใช้ Python ในการเปิดไฟล์แบบ **"Append"** (ต่อท้าย) เพื่อไม่ให้ข้อมูลเก่าหายไป

NotebookLM

แสดงข้อมูลประวัติย้อนหลัง (Historical View)

ให้ Python อ่านไฟล์ weather.csv (อ่านจากบรรทัดล่างสุดขึ้นมา) ดึงข้อมูล 10 รายการล่าสุด

Latest 10 Records

Date	Time	Temp	Hum
2023-11-14	15:45:30	32.1°C	68.5%
2023-11-14	15:40:30	32.0°C	68.2%
2023-11-14	15:35:30	31.8°C	68.0%
2023-11-14	15:30:30	31.7°C	67.8%
2023-11-14	15:25:30	31.5°C	67.5%
2023-11-14	15:20:30	31.4°C	67.2%
2023-11-14	15:15:30	31.2°C	67.0%
2023-11-14	15:10:30	31.0°C	66.8%
2023-11-14	15:05:30	30.8°C	66.5%
2023-11-14	15:00:30	30.6°C	66.2%

Takeaway: ตอนนี้ Dashboard ของเรามีทั้ง **"ข้อมูลปัจจุบัน"** (Real-time) และ **"ประวัติย้อนหลัง"** (History)

NotebookLM

Checklist ทดสอบระบบก่อนเริ่มโครงงาน

✅ ESP32
เชื่อมต่อ Wi-Fi สำเร็จ
และเซ็นเซอร์อ่านค่าได้จริง

✅ Raspberry Pi
Python requests สามารถดึง JSON
จาก ESP32 ได้โดยไม่เกิด Error

✅ Storage
ไฟล์ weather.csv ถูกสร้างขึ้นแ
ละมีข้อมูลเพิ่มขึ้นทุก 5 วินาที

✅ User Interface
สามารถเปิด Dashboard ผ่าน
Web Browser ของสมาร์ตโฟน
ในเครือข่ายเดียวกันได้

NotebookLM

Final Project: Weather Monitoring IoT System (40 นาที)

Mission Brief

Advanced Challenges (แบบฝึกหัดท้าทาย)

- ☑ เพิ่มข้อมูล "ชื่อห้อง" (เช่น ห้อง Server, แปลงผัก) ลงใน Dashboard
- ☑ UI Logic: เปลี่ยนสีตัวเลขเป็นสีแดงเมื่ออุณหภูมิสูงกว่า 35°C

NotebookLM

การนำเสนอผลงาน (Pitch & Demo)

เวลานำเสนอ 5 นาที / กลุ่ม

Q1: โครงสร้างและการทำงาน

อธิบาย Architecture และวิธีการเชื่อมต่อระบบ

Q2: Live Demo

สาธิตการทำงานจริง (ค่าอุณหภูมิเปลี่ยนเมื่อเซ็นเซอร์ถูกกระตุ้น)

Q3: ปัญหาและการแก้ปัญหา (Debugging)

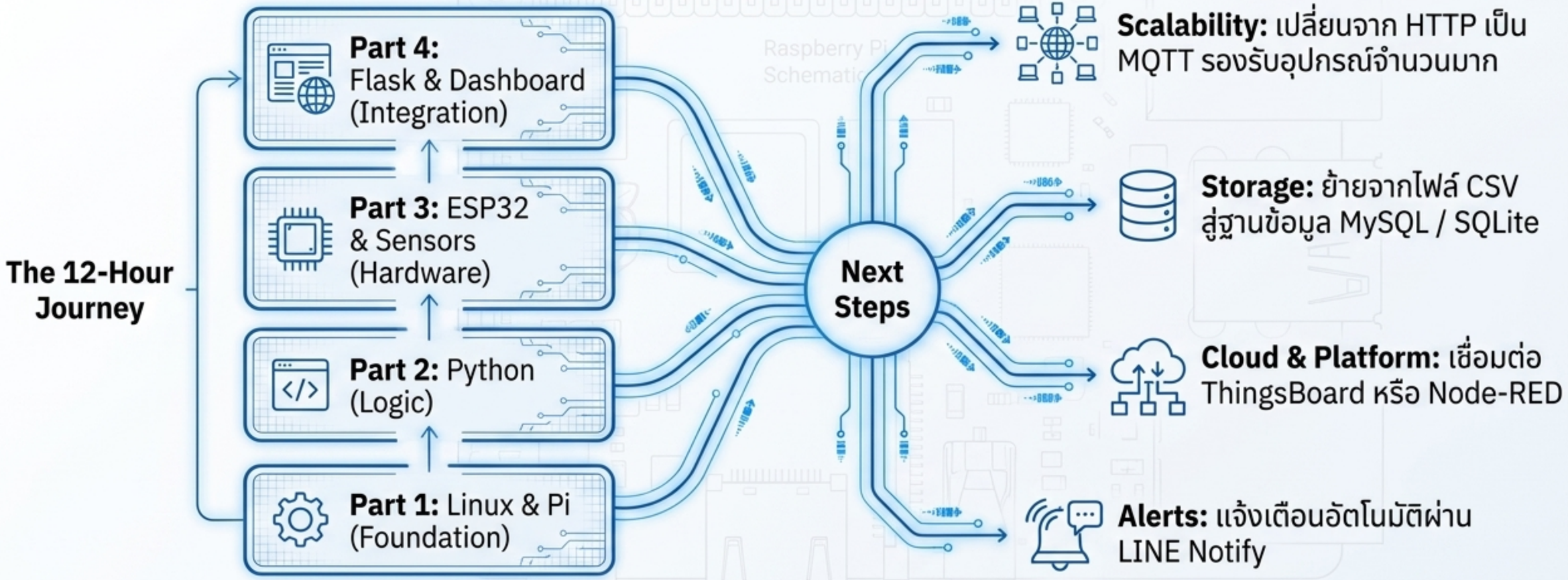
อุปสรรคที่พบระหว่างทางและวิธีแก้ไข

Q4: แนวทางต่อยอด (Future Scope)

ไอเดียในการนำไปประยุกต์ใช้กับ Smart Home หรือ Smart Farm

NotebookLM

บทสรุปหลักสูตร: จากพื้นฐานสู่ระบบ AIoT



คุณพร้อมแล้วสำหรับการพัฒนาระบบ Smart City และ AIoT ในระดับอุตสาหกรรม

NotebookLM